

Dokumentacja projektu na RSI - RunFasta

Tomasz Jarmoc - 113305

1. Cel projektu i opis funkcjonalności

RunFasta to aplikacja webowa, której głównym celem jest śledzenie postępów treningowych w biegach zarówno po stronie biegacza jak i jego trenera. Projekt składa się z następujących funkcjonalności:

1.1 Rejestracja konta

Rejestracja

Wybierz rolę

▼

Zarejestruj się

[Masz już konto? Zaloguj się](#)

Wypełnij wszystkie pola!

Konto w aplikacji możemy założyć podając swoją nazwę użytkownika, hasło jak i wybraną przez siebie rolę (trenera lub biegacza).

1.2 Logowanie

Logowanie

Zaloguj

[Nie masz konta? Zarejestruj się](#)

Podaj login i hasło!

Po założeniu konta należy się do niego zalogować, podając swoją nazwę użytkownika i hasło.

1.3 Panel biegacza

Każda rola w aplikacji ma swój oddzielny panel z oddzielnymi funkcjonalnościami. W przypadku biegacza są to 3 oddzielne zakładki:

1.3.1 Dzisiejsze treningi

Panel Biegacza: biegacz1 Wyloguj

Dzisiejsze treningi

Moja Historia treningów

Statystyki treningów i kontakt z trenerem

Zadane treningi od trenera na dziś

Dziś brak planów treningowych. Możesz odpocząć lub wpisać trening własny.

Zaraportuj dzisiejszy bieg (trening własny)

Dystans (km)

Czas (min)

Tempo i prędkość biegu (uzupełniają się automatycznie):

Tempo (min/km)

Prędkość (km/h)

Jak się biegło?

Zapisz Trening

Wprowadź dystans i czas!

W tej zakładce biegacz ma możliwość obejrzenia treningów zadanych przez trenera na dzień obecny, może taki trening również zaraportować jako wykonany. W przypadku, jeżeli biegacz miałby własną inwencję twórczą i chciałby dodać własny trening, również może to zrobić wraz z podanymi przez siebie parametrami jak dystans, czas i ogólne odczucia z treningu.

1.3.2 Historia treningów wraz z zapisem do PDFa

Panel Biegacza: biegacz1

Wyloguj

Dzisiejsze treningi

Moja Historia treningów

Statystyki treningów i kontakt z trenerem

Filtrowanie historii treningów

Zakres dat:
Start Date → 2026-06-13

Eksport PDF:
Pobierz raport (PDF)

Lp.	ID bazy	Data	Dystans (km)	Czas (min)	Tempo	Typ	Status	Komentarz / Notatki / Interwały
1	1	2026-06-12	5	30		Własny	Ukończono	super
2	3	2026-06-12	5	27	5:24 min/km	Zadanie	Oczekuje	
3	4	2026-06-12	5	27	5:4 min/km (11.11 km/h)	Zadanie	Ukończono	Interwał 3.0x1000.0m (przerwa: 2 min marsz) Powtórzenia dość szybko, po 5'00"/KM
4	5	2026-06-12	6	30	5 min/km (12 km/h)	Własny	Ukończono	szybko, równe tempo

Usunij wpis (podaj ID z tabeli)

ID

Usunij

Podaj ID!

W tej zakładce biegacz może przeglądać wcześniej wykonane treningi zarówno zakończone jak i oczekujące. Obok rozpiski jest też możliwość pobrania PDFa z wyciągiem z treningów jak i usunięciem danego treningu po jego ID.

1.3.3 Statystyki treningów i czat z trenerem

Panel Biegacza: biegacz1

Wyloguj

Dzisiejsze treningi

Moja Historia treningów

Statystyki treningów i kontakt z trenerem

Twoje statystyki

Start Date → 2026-06-13

Łączny dystans

16.0 km

Liczba treningów

3

Komunikacja z trenerem

cześć

2026-06-12 18:38

halo

2026-06-12 19:31

witaj trenerze

2026-06-12 19:32

Napisz do trenera

Wyślij

W ostatnim panelu biegacz ma możliwość spojrzenia na zbiorcze statystyki ze swoich biegów za wybrany okres czasu jak i wymiany spostrzeżeń bezpośrednio ze swoim trenerem w czasie rzeczywistym na czacie.

1.4 Panel trenera

Panel trenera również składa się z 3 oddzielnych stron, którymi są:

1.4.1 Zarządzanie grupą podopiecznych

Panel Trenera: trener1

Wyloguj

Panel grupy podopiecznych i statystyki

Zadawanie treningów podopiecznym

Czat z podopiecznymi i raportowanie treningów do PDF

Zarządzanie grupą

Wybierz biegacza do dodania

Dodaj do Grupy

biegacz1 (ID: 2)

biegacz2 (ID: 3)

Statystyki biegacza dla wybranego zakresu dat

Wybierz biegacza

Start Date → 2026-06-13

Dystans łączny

0.0 km

Suma treningów

0

Dzisiejsze plany twoich podopiecznych

biegacz1

Brak planów na dziś

biegacz2

Brak planów na dziś

W pierwszym panelu trener może dodawać i usuwać ze swojej grupy biegaczy, zobaczyć statystyki z jego treningów z danego okresu czasu jak i podejrzeć plany treningowe na obecny dzień dla swoich podopiecznych.

1.4.2 Zadawanie treningów podopiecznym

The screenshot shows the 'Zadawanie treningów podopiecznym' (Assigning training to subordinates) panel. At the top, there's a header 'Panel Trenera: trener1' and a 'Wyloguj' button. Below the header are three tabs: 'Panel grupy podopiecznych i statystyki', 'Zadawanie treningów podopiecznym' (which is active), and 'Czat z podopiecznymi i raportowanie treningów do PDF'. The main content area is titled 'Zadaj nowy trening podopiecznemu'. It contains a dropdown menu 'Wybierz biegacza', two input fields for 'Dystans (km)' and 'Planowany Czas (min)', a date selector showing '2026-06-13', and a section for 'Docelowe tempo i prędkość (uzupełniają się automatycznie):' with fields for 'Tempo (min/km)' and 'Prędkość (km/h)'. There is a radio button for 'Trening interwałowy' and a text area for 'Dodatkowe zalecenia trenera'. At the bottom, there are two buttons: 'Akceptuj i wyślij trening podopiecznemu' (green) and 'Wybierz biegacza i podaj dystans!' (orange).

W tym panelu trener może wybrać jednego ze swoich podopiecznych i zadać mu trening z wybranymi przez siebie parametrami. Jeżeli wybierze trening interwałowy, może również wybrać tempo i przerwy tychże interwałów oraz dodać komentarz do treningów.

1.4.3 Wyciąganie raportów z treningów do PDF i czat

The screenshot shows the 'Czat z Biegaczem' (Chat with runner) panel. It has the same header and tabs as the previous panel. The main content area is titled 'Czat z Biegaczem'. It contains a dropdown menu 'Wybierz biegacza, by pisać', a text area for 'Wybierz biegacza z listy, aby wyświetlić historię rozmów.', a text input field for 'Napisz wiadomość', and a 'Wyślij' button. On the left side, there's a sidebar titled 'Uzyskaj raporty treningów w PDFie' with a dropdown menu 'Wybierz biegacza' and three buttons: 'Pobierz Plan Wybranego' (blue), 'Pobierz Plany Wszystkich' (grey), and 'Wybierz najpierw biegacza!' (orange).

W ostatnim panelu trener wyszukując danego podopiecznego może wyciągnąć raport wraz z zapisem treningów do PDFa jak i wyciągnąć raport zbiorczy dla wszystkich swoich podopiecznych. Można tu również porozmawiać na bieżąco z wybranym podopiecznym odnośnie jego treningów.

2. Instrukcja instalacji, uruchomienia i obsługi aplikacji

Aby móc korzystać z tejże aplikacji należy pobrać kod projektu, a następnie wykonać następujące kroki:

1. Zainstalować środowisko Python w wersji 3.10+

2. W terminalu uruchomić komendę:
`pip install fastapi uvicorn sqlalchemy fpdf2 pydantic requests dash dash-bootstrap-components pandas urllib3`
3. Wygenerować certyfikaty służące do komunikacji HTTPS/SSL poprzez następującą komendę:
`openssl req -x509 -newkey rsa:4096 -keyout key.pem -out cert.pem -sha256 -days 365 -nodes`
4. Uruchomić 2 terminale
5. W pierwszym z nich trzeba uruchomić serwer komendą:
`python -m uvicorn main:app --host 127.0.0.1 --port 8000 --ssl-certfile cert.pem --ssl-keyfile key.pem`
6. W drugim z nich (może to być inny komputer) uruchomić klienta komendą:
`python app.py`

3. Opis endpointów HTTP (najważniejsze wybrane)

Poniżej znajduje się specyfikacja techniczna najważniejszych endpointów wystawionych przez serwer RESTful Web Service (main.py). Wszystkie endpointy wymagają przesłania nagłówka autoryzacyjnego Authorization: Basic <credentials>.

UWAGA:

Pełna rozpiska wraz z całościową dokumentacją endpointów jest tworzona automatycznie przez bibliotekę FastAPI(Swagger).

Aby mieć dostęp do tej dokumentacji po uruchomieniu servera należy w oknie przeglądarki wpisać: **`https://127.0.0.1:8000/docs#/`**.

3.1. Weryfikacja tożsamości i pobranie profilu (GET /me)

Opis: Służy do uwierzytelnienia klienta na starcie aplikacji i określenia jego uprawnień (rola COACH lub RUNNER).

Metoda: GET

URL: `https://127.0.0.1:8000/me`

Nagłówek żądania (Request Header):

Authorization: Basic dHJlbmVyMToxMjM=

JSON

```
{
  "id": 2,
  "username": "biegacz1",
  "role": "RUNNER",
  "coach_id": 1
}
```

Statusy odpowiedzi:

200 OK – Autoryzacja pomyślna, zwrócono profil.

401 Unauthorized – Brak lub błędne dane uwierzytelniające.

3.2. Pobieranie listy treningów z implementacją HATEOAS (GET /workouts)

Opis: Pobiera listę wszystkich treningów (trener pobiera treningi swoich podopiecznych, biegacz pobiera własne). Zwraca metadane(HATEOAS)

Metoda: GET

URL: <https://127.0.0.1:8000/workouts>

JSON

```
[
  {
    "id": 14,
    "runner_id": 2,
    "distance": 12.0,
    "time_minutes": 60,
    "note": "Trening progowy",
    "is_planned": true,
    "workout_date": "2026-06-12",
    "completed": false,
    "target_pace": "5.0 min/km (12.0 km/h)",
    "is_interval": true,
    "interval_repeats": 5,
    "interval_distance_meters": 1000,
    "interval_recovery": "3 min trucht",
    "links": [
      {
        "rel": "self",
        "href": "/workouts/14"
      }
    ]
  }
]
```

Statusy odpowiedzi: 200 OK, 401 Unauthorized.

3.3. Dynamiczne filtrowanie statystyk biegacza (GET /stats/runner/{runner_id})

Opis: Pobiera statystyki przebiegu wybranego biegacza. Obsługuje opcjonalne parametry zapytania służące do dynamicznego filtrowania zakresu dat.

Metoda: GET

URL: https://127.0.0.1:8000/stats/runner/{runner_id}

Parametry query (opcjonalne):

start_date (Format: YYYY-MM-DD)

end_date (Format: YYYY-MM-DD)

JSON

```
{
  "total_distance": 45.5,
  "count": 6
}
```

Statusy odpowiedzi:

200 OK – Dane pomyślnie przetworzone.

403 Forbidden – Próba dostępu do danych biegacza nieprzypisanego do trenera.

3.4. Zadawanie treningu podopiecznemu (POST /workouts/plan)

Opis: Pozwala trenerowi na zaplanowanie biegu standardowego bądź interwałowego z docelowym tempem i strukturą powtórzeń.

Metoda: POST

URL: <https://127.0.0.1:8000/workouts/plan>

Tylko rola COACH

JSON

```
{
  "runner_id": 2,
  "distance": 8.0,
  "time_minutes": 40,
  "note": "Interwały tempowe na bieżni",
  "workout_date": "2026-06-15",
  "target_pace": "5.0 min/km (12.0 km/h)",
  "is_interval": true,
  "interval_repeats": 8,
  "interval_distance_meters": 400,
  "interval_recovery": "2 min marsz"
}
```

Statusy odpowiedzi:

200 OK – Plan pomyślnie utworzony w bazie danych.

404 Not Found – Biegacz o podanym ID nie istnieje.

3.5. Pobieranie kompleksowego raportu PDF biegacza (GET /report/pdf/runner/{runner_id})

Opis: Generuje i wysyła strumień binarny pliku PDF zawierający zestawienie wszystkich zaplanowanych oraz zrealizowanych treningów.

Metoda: GET

URL: `https://127.0.0.1:8000/report/pdf/runner/{runner_id}`

Wymagana autoryzacja: Tak

Odpowiedź (HTTP Headers & Payload):

Nagłówek: Content-Type: application/pdf

Payload: Strumień binarny pliku PDF (wyświetlany w przeglądarce lub pobierany na dysk).

3.6. Rejestracja nowego konta (POST /register)

Metoda: POST

URL: `https://127.0.0.1:8000/register`

JSON

```
{
  "username": "nowy_biegacz",
  "password": "haslo_uzytkownika",
  "role": "RUNNER"
}
```

Opis działania: Sprawdza, czy nazwa użytkownika jest unikalna oraz czy przesłana rola odpowiada dozwolonym wartościom (COACH lub RUNNER). Po walidacji tworzy nowy rekord w bazie.

3.7. Pobranie wiadomości czatu z danego wątku wiadomości (GET /messages/thread/{other_id})

Metoda: GET

URL: `https://127.0.0.1:8000/messages/thread/{other_id}`

Opis działania: Pobiera historię rozmowy pomiędzy zalogowanym użytkownikiem a adresatem wskazanym w ścieżce zapytania (other_id). Zapytanie SQL filtruje wiadomości w obie strony (nadawca -> odbiorca oraz odbiorca -> nadawca) i sortuje je chronologicznie.

JSON

```
[
  {
    "id": 4,
    "sender_id": 1,
```



```
"receiver_id": 2,  
"text": "Cześć! Jak poszły dzisiejsze interwały?",  
"created_at": "2026-06-12T21:15:30.123456"  
},  
{  
  "id": 5,  
  "sender_id": 2,  
  "receiver_id": 1,  
  "text": "Bardzo dobrze, tętno w normie.",  
  "created_at": "2026-06-12T21:17:10.654321"  
}  
]
```

Statusy HTTP: 200 OK, 401 Unauthorized.

4. Opis najważniejszych funkcji i zależności projektu.

Backend:

4.1. get_current_user

Argumenty:

credentials: HTTPBasicCredentials (dane z nagłówka autoryzacyjnego HTTP Basic rozkodowane automatycznie przez FastAPI).

db: Session (sesja połączenia z bazą danych).

Co zwraca: Obiekt użytkownika z bazy danych (models.User) w przypadku sukcesu lub rzuca wyjątek HTTPException o statusie 401 Unauthorized (niepoprawne hasło lub login).

Co robi: Pełni rolę autoryzacji (Request Filter). Odczytuje dane uwierzytelniające, zabezpiecza hasła przed atakami typu timing attack przy pomocy bezpiecznego porównania (secrets.compare_digest z pythonowej biblioteki secrets) i decyduje o dopuszczeniu żądania do chronionego endpointu.

4.2. _pdf_row_wrapped

Argumenty:

pdf: FPDF (instancja dokumentu PDF biblioteki FPDF).

cols: List[Tuple[int, Any]] (lista krotek reprezentujących strukturę kolumn wiersza

tabeli: szerokość kolumny oraz jej tekst).

Co zwraca: Nic (None). Metoda modyfikuje przekazaną instancję dokumentu PDF.

Co robi: Odpowiada za składanie wielolinijkowych wierszy tabeli w pliku PDF. Dzieli długie ciągi tekstowe na pojedyncze linie za pomocą modułu textwrap, oblicza

wysokość najwyższej komórki wiersza, zapobiega ucinaniu tekstu na krawędziach kolumn i dba o to, by obramowania siatki tabeli rysowały się na pełną wysokość wiersza. Obsługuje też automatyczne przejście do nowej strony przy przepełnieniu.

4.3. register (Endpoint POST /register)

Argumenty:

data: UserRegister (obiekt schematu Pydantic zawierający login, hasło i rolę).

db: Session (sesja bazy danych).

Co zwraca: Nowo utworzonego użytkownika zmapowanego do schematu UserResponse lub rzuca wyjątek HTTPException przy zajętych loginie lub przy błędnej roli.

Co robi: Odpowiada za rejestrację konta. Waliduje unikalność loginu, sprawdza poprawność zdefiniowanej roli i zapisuje rekord użytkownika w bazie relacyjnej SQLite.

4.4. plan_workout (Endpoint POST /workouts/plan)

Argumenty:

plan: WorkoutPlanCreate (schemat Pydantic zawierający parametry treningu i interwałów).

user: User (obiekt zalogowanego trenera wstrzyknięty przez filtr autoryzacji).

db: Session (sesja bazy danych).

Co zwraca: Zaplanowany trening zmapowany do schematu WorkoutResponse lub rzuca wyjątek HTTPException (403 Forbidden jeśli wywołującym nie jest trener).

Co robi: Umożliwia trenerowi planowanie biegów dla podopiecznych. Obsługuje zapis parametrów standardowych oraz zaawansowanych danych interwałowych (liczba powtórzeń, odległość powtórzenia, czas przerwy).

4.5. get_runner_stats (Endpoint GET /stats/runner/{runner_id})

Argumenty:

runner_id: int (klucz główny biegacza ze ścieżki URL).

start_date: Optional[date] oraz end_date: Optional[date] (opcjonalne daty filtracji przekazywane w Query Parameters).

user: User (obiekt zalogowanego użytkownika).

db: Session (sesja bazy danych).

Co zwraca: Słownik JSON zawierający sumaryczny dystans (`total_distance`) oraz liczbę ukończonych biegów (`count`) w zadanym okresie.

Co robi: Dynamicznie agreguje (sumuje) dystans i liczbę zrealizowanych treningów dla danego biegacza z uwzględnieniem przesłanego zakresu czasu, sprawdzając uprzednio uprawnienia dostępu do danych (ochrona prywatności).

FRONTEND

4.6. `sync_pace_speed_coach`

Argumenty:

`dist`: float (dystans z pola wprowadzania danych).

`time`: int (czas z pola wprowadzania).

`pace`: float (tempo w min/km).

`speed`: float (prędkość w km/h).

Co zwraca: Krotkę składającą się z trzech wartości: (`tempo_min_km`, `prędkość_km_h`, `komponent_alertu_ostrzegawczego`).

Co robi: Synchronizuje i matematycznie przelicza parametry biegowe w formularzu. Na podstawie dystansu i czasu oblicza tempo i prędkość. Przy ręcznej modyfikacji tempa wylicza prędkość (i na odwrót). Wykonuje walidację bezpieczeństwa – jeśli prędkość wykracza poza zakres 3 km/h – 20 km/h, blokuje formularz i zwraca czerwony alert z błędem.

4.7. `load_runners_list`

Argumenty:

`auth`: dict (słownik z danymi zalogowanego trenera z `session-auth.data`).

`_`: int (wartość triggera aktualizacji z `coach-update-trigger.data`).

Co zwraca: Zestaw sześciu wartości zawierający wyrenderowaną listę biegaczy HTML (`dbc.ListGroup`) oraz listy opcji w formacie słownikowym dla wszystkich dropdownów w panelu trenera.

Co robi: Pobiera asynchronicznie z serwera za pomocą biblioteki `requests` listy przypisanych oraz wolnych biegaczy. Inicjalizuje listę typu dropdown zawierającą podopiecznych trenera.

4.8. `update_runner_history_table`

Argumenty:

`auth`: dict (dane sesji biegacza).

`_`: int (wartość wyzwalacza aktualizacji z `runner-update-trigger.data`).

`start_date`: str oraz `end_date`: str (tekstowe reprezentacje dat z kalendarza).

Co zwraca: Komponent tabeli Bootstrap (dbc.Table) z pełną historią lub informację tekstową o braku rekordów.

Co robi: Pobiera wszystkie zaplanowane i własne treningi biegacza z API. Konwertuje strukturę JSON do obiektu Pandas DataFrame, przeprowadza filtrację wierszy w zadanym zakresie dat po stronie klienta, wylicza lokalną kolumnę porządkową Lp., mapuje statusy treningów (Oczekuje/Ukończono) i renderuje wynikową tabelę.

4.9. load_runner_chat

Argumenty:

coach_id: int (ID trenera odczytane ze Store).

n: int (licznik interwału czasowego runner-chat-refresh wywoływanego co 5s).

_: int (wartość wyzwalacza zmian z runner-update-trigger.data).

auth: dict (dane sesji).

Co zwraca: Listę komponentów typu div reprezentującą sformatowane dymki rozmowy czatu.

Co robi: Odpowiada za obsługę czatu w czasie rzeczywistym. Odpytuje co 5 sekund endpoint /messages/thread/{id} przez API, układa wiadomości po lewej stronie (trener) lub prawej stronie (biegacz), dobiera odpowiednie kolory tła i generuje okno rozmowy. Uśpiona, gdy biegacz przebywa w innej zakładce.